

Chapitre 1 : Scalable Systems

1. Rappel sur la normalisation

La normalisation organise les données pour :

- réduire la redondance,
- éviter les incohérences,
- faciliter la maintenance.

Formes normales :

- **1NF** : valeurs atomiques (une seule valeur par colonne).
- **2NF** : dépendance totale des colonnes non-clés à la clé primaire complète.
- **3NF** : absence de dépendance transitive entre colonnes non-clés.

Objectif global : bases de données plus cohérentes, claires et faciles à maintenir.

2. Systèmes évolutifs (Scalable Systems)

Un système scalable peut gérer une augmentation de charge (utilisateurs, données, requêtes) en ajoutant des ressources, **sans dégradation significative des performances**.

3. Caractéristiques principales

- Supporte plus d'utilisateurs et de données.
- Ajout de ressources sans refonte complète du système.
- Performances stables malgré la charge.
- Capacité d'adaptation aux besoins futurs.

4. Importance de la scalabilité

- Accompagne la croissance de l'entreprise.
- Réduit les coûts par ajout progressif de ressources.
- Garantit performance et fiabilité.
- Gère les pics de trafic imprévus.

5. Stratégies pour scaler une base de données

1. Indexation

- Accélère les lectures.
- Peut ralentir les écritures.
- Nécessite un bon choix des colonnes indexées.

2. Vues matérialisées

- Données pré-calculées pour des requêtes rapides.
- Rafraîchissement nécessaire.
- Très utilisées en BI.

3. Dénormalisation

- Ajout volontaire de redondance.
- Moins de jointures, requêtes plus rapides.
- Exemple : réseaux sociaux.

4. Scalabilité verticale

- Ajouter CPU, RAM, stockage à un serveur existant.
- Simple à mettre en œuvre.
- Limite : point de défaillance unique.

5. Caching

- Stockage temporaire en mémoire (Redis, Memcache).
- Réduit la charge sur la base.
- Nécessite gestion de la cohérence du cache.

6. Réplication

- Copies de la base sur plusieurs serveurs.
- Améliore disponibilité et lectures.
- Synchrone (cohérence forte) ou asynchrone (meilleures performances).

7. Sharding

- Découpage de la base en plusieurs fragments (shards).
- Très efficace pour grandes bases.
- Complexe à concevoir et maintenir.

Conclusion

La scalabilité repose sur un **équilibre entre performance, coût et complexité**. Combiner intelligemment ces stratégies permet de construire des bases de données capables de supporter la croissance et les fortes charges.

Chapitre 2 : Design d'un système scalable (de 0 à des millions d'utilisateurs)

1. Scalabilité verticale

- Ajout de ressources matérielles (CPU, RAM) à un seul serveur.
- Simple à mettre en place.
- Limitation physique stricte.
- Insuffisante pour des systèmes à très grande échelle.

2. Scalabilité horizontale

- Ajout de plusieurs serveurs dans un même système.
- Méthode clé pour les applications à grande échelle.
- Les requêtes sont réparties entre les serveurs grâce à un **load balancer**.

3. DNS (Domain Name System)

- Traduit les noms de domaine (ex. netflix.com) en adresses IP.
- Permet un accès simple aux sites web sans mémoriser les IP.

Résolution DNS inverse (Reverse DNS) :

- Convertit une adresse IP en nom de domaine (via enregistrements PTR).
- Utilisée pour la sécurité, les mails et l'administration réseau.

4. Rôle du Load Balancer

- Point d'entrée unique pour les utilisateurs.
- Répartit les requêtes vers les serveurs les moins chargés.
- Avantages :
 - Meilleure répartition de la charge.
 - Sécurité accrue (serveurs internes masqués).
 - Haute disponibilité.

5. Problème d'une base de données unique et réplication

- Une seule base = point de panne unique.
- **Solution : réplication**
 - **Master DB** : écritures.
 - **Slave DB** : lectures.
 - Synchronisation des données entre instances.
- Améliore performances, disponibilité et tolérance aux pannes.

6. Load balancer pour bases de données

- Répartit automatiquement les requêtes de lecture entre les bases répliquées.
- Évite de surcharger un seul replica.
- Améliore la scalabilité horizontale des bases de données.

7. Architecture scalable complète

Chaîne globale :

Utilisateur → DNS → Load Balancer → Serveurs applicatifs → Base de données

- Supporte un grand nombre d'utilisateurs.
- Assure rapidité, fiabilité et haute disponibilité.

8. Problèmes de performance et solutions

- **Panne du Master DB** :
 - Solution : promotion automatique d'un Slave en Master.
- **Requêtes base de données coûteuses** :
 - Solution : mise en place d'un **cache** entre serveur et base.

9. Rôle du cache

- Stocke temporairement les données fréquemment utilisées.
- Réduit la charge sur la base de données.

- Accélère les temps de réponse.
- Exemple : cache clé-valeur (Redis, Memcached).

10. Cache DNS

- Le cache DNS évite de refaire des résolutions complètes.
- Réduit la latence et la charge réseau.
- Utilisé localement et par les serveurs DNS.

11. Aspects importants du cache

- **Eviction Policy** : quelles données supprimer quand le cache est plein.
- **Expiration Policy** : durée de validité des données.
- **Consistency Requirements** : cohérence entre cache et base de données.

Chapitre 3 : Microservices Architecture

1. Deux styles d'architecture

Architecture monolithique

- Application développée en un seul bloc.
- Une seule technologie et un seul déploiement.
- Organisation en couches :
 - UI (présentation)
 - Business Logic (logique métier)
 - Data Access Layer (DAO)
 - Base de données unique
- **Avantages** : simple à développer, tester et déployer au départ.
- **Inconvénients** : difficile à maintenir, à faire évoluer et à scaler ; redéploiement global à chaque changement.

Architecture microservices

- Application découpée en services indépendants.
- Chaque microservice :
 - Gère une fonctionnalité précise
 - Possède sa propre base de données
 - Communique via des API
- **Avantages** : scalabilité indépendante, déploiement séparé, équipes autonomes.
- **Inconvénients** : complexité accrue (réseau, données distribuées, DevOps).

2. Organisation des architectures

Monolithique (en couches)

- Couche Web : UI, Web services, GraphQL.
- Couche Business : règles métier.
- Couche DAO : accès aux données (Hibernate, Spring Data, JPA).
- Couche Base de données : stockage centralisé.

Microservices moderne

- UI (Web / Mobile / Desktop) → API
- Services métier indépendants
- DAO / Repositories pour l'accès aux données
- Utilisation de :
 - Base de données
 - Cache
 - Moteur de recherche (ex. Elasticsearch)
 - Event Bus pour communication asynchrone
- Favorise scalabilité, résilience et maintenabilité.

3. Bonnes pratiques en microservices

- **Entités :**
 - Mauvaise pratique : exposer les entités partout.
 - Bonne pratique :
 - Entités utilisées en couche métier et persistance
 - DTOs utilisés en couche Web
 - Mappers pour convertir Entity ↔ DTO
- DAO : lien entre logique métier et base de données.
- Exemple de modules :
 - E-commerce, inventaire, clients, facturation, commandes.

4. Architecture microservices typique

- Clients → **API Gateway** (point d'entrée unique)
- Services centraux :
 - API Gateway : routage, sécurité, load balancing
 - Config Service : configuration centralisée
 - Discovery Service : localisation dynamique des services
- Chaque microservice :
 - API → Service → DTO/Mapper → Repository → Entity → DB
- **Database per Service** : une base par microservice.
- **Event Bus** (Kafka, RabbitMQ) : communication asynchrone.
- Patterns clés :
 - API Gateway
 - Service Discovery
 - Database per Service

- Event-Driven Architecture

5. API Gateway et configuration

- API Gateway :
 - Point d'entrée unique
 - Routage des requêtes
 - Équilibrage de charge
- Types :
 - Statique
 - Dynamique (avec Discovery Service)
- Configuration centralisée :
 - Un seul fichier de configuration partagé
 - Évite la duplication et facilite la maintenance.

6. Gestion de la configuration

- Configuration à froid : lecture statique, moins performante.
- Configuration à chaud : mise à jour dynamique en temps réel.
- Outils : Spring Cloud Config, Consul Config.

7. Développement avec Spring Cloud

- Chaque microservice correspond à un périmètre fonctionnel.
- Développement parallèle possible (équipes indépendantes).
- Exemple universitaire :
 - Microservices Étudiants, Classes, Notes, Modules, Salles.
- Avantage : rapidité globale de développement et scalabilité.

8. Démarrage et enregistrement des microservices

- Démarrage du microservice.
- Récupération de la configuration depuis le Config Service.
- Enregistrement auprès du Discovery Service.
- Choix technologique par service :
 - Python : IA / ML
 - Node.js : temps réel
 - Java : transactions

9. Docker et Kubernetes

- Docker :
 - Conteneurisation des applications.
- Kubernetes (K8s) :
 - Orchestration des conteneurs
 - Déploiement, scalabilité et gestion automatique

- Projet open source issu de Google.

10. Challenges et communication

Défis principaux

- Communication inter-services
- Données distribuées
- Orchestration et monitoring

Protocoles réseau

- Orientés connexion : TCP, FTP
- Non orientés connexion : UDP, IP
- Mode connecté : gestion de session
- Mode non connecté : paquets indépendants

Communication

- Synchrones : requête/réponse immédiate.
- Asynchrones : échanges différés via événements.

Chapitre 4 : RESTful Web Services

1. REST (Representational State Transfer)

- REST est un **style d'architecture** issu des principes du Web.
- Il repose sur un **ensemble de contraintes** appliquées progressivement pour obtenir une architecture cohérente.
- Approche **top-down** : on part des besoins globaux puis on ajoute des contraintes.
- Les contraintes REST guident naturellement l'architecture vers un système simple, scalable et interopérable.

2. Modèle vide (Null State)

- Système sans aucune contrainte architecturale.
- Aucune règle sur la structure ou les interactions.
- Le **World Wide Web** est considéré comme le Null State de REST.
- REST = Null State + contraintes successives.

3. Client – Serveur

- Séparation claire :
 - **Client** : interface utilisateur.
 - **Serveur** : données et logique métier.
- Avantages :
 - Portabilité des interfaces.
 - Scalabilité côté serveur.

- Évolution indépendante client / serveur.

4. Sans état (Stateless)

- Chaque requête contient toutes les informations nécessaires.
- Le serveur ne conserve aucun état entre les requêtes.
- Avantages :
 - Simplicité
 - Scalabilité
 - Fiabilité
- Inconvénients :
 - Surcoût réseau
 - Moins de contrôle côté serveur.

5. Cache

- Les réponses indiquent si elles sont **cachables** ou non.
- Réduit la charge serveur et améliore les performances.
- Améliore :
 - Performance
 - Scalabilité
 - Efficacité globale

6. Interface uniforme

Objectif : découpler client et serveur.

4 contraintes clés :

1. **Identification des ressources**
 - Chaque ressource a une URI unique.
2. **Manipulation via des représentations**
 - JSON, XML, HTML, etc.
3. **Messages auto-descriptifs**
 - Le message contient toutes les infos nécessaires.
4. **Hypermédia (HATEOAS)**
 - Les liens guident les actions possibles.

Interface standardisée, évolutive et interopérable

Inconvénient : moins d'optimisations spécifiques

7. Système en couches

- Chaque couche ne connaît que la couche adjacente.
- Avantages :
 - Réduction de la complexité
 - Sécurité renforcée
 - Indépendance des composants

- Inconvénient :
 - Latence et surcoût (atténués par le cache)

8. Code à la demande (optionnel)

- Le serveur peut envoyer du code exécutable au client.
- Avantage : extensibilité.
- Inconvénient : perte de visibilité et de contrôle.
- Contrainte **optionnelle** en REST.

9. Éléments de données REST

- **Ressource** : concept ciblé (ex. article, utilisateur).
- **Identifiant** : URL / URI.
- **Représentation** : HTML, JSON, image, etc.
- **Métadonnées** :
 - Type de média
 - Date de modification
- **Données de contrôle** :
 - cache-control
 - if-modified-since

10. Représentations REST

- Les actions se font via des **représentations**.
- Une représentation = données + métadonnées.
- Peut contenir :
 - Données
 - Métadonnées de représentation
 - Métadonnées de ressource
 - Données de contrôle
- Une même ressource peut avoir plusieurs représentations selon le client.

11. Connecteurs REST

- **Client** : accès aux ressources (ex. libwww).
- **Serveur** : exposition des ressources (ex. Apache API).
- **Cache** : stockage temporaire (ex. navigateur, CDN).
- **Résolveur** : résolution de noms (DNS).
- **Tunnel** : sécurisation / relais (SSL, SOCKS).

12. Ressources REST

- Toute entité accessible sur le Web.
- Correspond à un concept métier.
- Exemples :
 - Article d'actualité

- Température à un instant donné
- Résultat de recherche Google
- Déclaration fiscale

13. Représentation REST (détail)

- État temporaire de la ressource au moment de la requête.
- Composée :
 - Flux binaire
 - Métadonnées (type, validation, sécurité)
- Plusieurs formats possibles pour une même URI :
 - HTML (humain)
 - JSON / XML (machine)

14. Mapping CRUD → HTTP

- **Create** → POST
- **Retrieve** → GET
- **Update** → PUT
- **Delete** → DELETE

Chapitre 5 : Inversion de Contrôle (IoC) et Injection de Dépendances

Principe fondamental

- Concevoir des applications **fermées à la modification** mais **ouvertes à l'extension**.
- Objectif : faciliter l'évolution, la maintenance et la scalabilité sans toucher au code existant.

1. Structure générale d'une application

Couches principales

- **Frontend**
 - Web : HTML, CSS, JavaScript
 - Desktop : Java AWT, Swing, JavaFX
 - Mobile : frameworks mobiles
- **Backend**
 - Technologies : PHP, Java EE, Node.js, .NET, Python
 - Frameworks Java EE : Spring, Spring Security, Hibernate
- **Frameworks Frontend**
 - CSS : Bootstrap
 - JS : Angular, Vue.js, React
- **Machine Learning**
 - Python

Concepts clés

- **Besoin fonctionnel** : besoin métier.
- **Besoin technique** : choix des technologies.
- **Déploiement en production** : mise en service réelle.

Problèmes possibles

- Mauvaise conception
- Mauvais développement
- Mauvais déploiement (architecture inadaptée)

Exemples de systèmes performants

- Google : milliards de recherches
- ChatGPT : millions de requêtes/jour
- Netflix : streaming haute performance

2. Netflix : aspects techniques

- **Technologies**
 - Node.js pour client et backend (simplicité, rapidité)
 - Java pour le cœur métier
- **Objectifs**
 - Maîtriser la technologie
 - Répondre aux besoins fonctionnels
 - Gérer la montée en charge
- **Solution**
 - Architectures **scalables** et **élastiques**

3. Scalabilité et Kubernetes

Types de scalabilité

- **Verticale** : ajouter CPU et RAM.
- **Horizontale** : multiplier les instances avec un load balancer.

Conteneurs et orchestration

- Applications exécutées dans des conteneurs (Docker).
- **Kubernetes** :
 - Surveille les conteneurs
 - Redémarre en cas de panne
 - Ajuste automatiquement le nombre d'instances
- Architecture **élastique** et **hautement disponible** (clusters).

Performance

- Dépend :
 - De l'architecture applicative
 - De l'architecture de déploiement

4. Scalabilité verticale et technologies

- **PHP**
 - Modèle multi-thread
 - Un thread par requête → limites techniques
- **Node.js**
 - Single-thread
 - E/S non bloquantes
 - Gère un grand nombre de requêtes avec peu de ressources
- **Conclusion**
 - Node.js est plus adapté à la scalabilité verticale que PHP.

5. Application fermée à la modification, ouverte à l'extension

Principe

- Le code existant ne doit pas être modifié.
- Les évolutions se font par **extensions**, **modules** ou **plugins**.

Avantages

- Maintenance plus simple
- Réduction des risques de régression
- Possibilité de TMA par des tiers

Risques de modification directe

- Bugs de régression
- Code difficile à comprendre
- Maintenance coûteuse sans tests

Exemples d'ERP

- SAP, Oracle, Odoo, Sage, Microsoft Dynamics, Zoho, Cegid, NetSuite, etc.

Chapitre 6 :SOA in Practice

1. Motivation

- Les systèmes modernes sont **grands, distribués et hétérogènes**.
- Ils doivent être **flexibles** et **communiquer facilement**.
- **SOA** permet de mieux gérer cette complexité.
- Principe clé (inspiré du darwinisme) :
 - Ce ne sont pas les systèmes les plus forts qui survivent, mais ceux qui **s'adaptent le mieux au changement**.

2. Caractéristiques des grands systèmes distribués

- **Gestion de l'existant (legacy) :**
 - Impossible de tout reconstruire.
 - SOA s'intègre progressivement en respectant la rétro-compatibilité.
- **Transformation continue :**
 - SOA n'est pas un projet ponctuel, mais une évolution structurelle.
- **Hétérogénéité :**
 - Multiples langages, plateformes et systèmes d'exploitation.
- **Complexité élevée :**
 - Difficile de comprendre l'impact des changements.
- **Imperfection inévitable :**
 - La perfection coûte trop cher.
 - Au-delà d'un seuil, l'effort augmente plus vite que les bénéfices.

Courbe « hockey stick »

- Complexité faible à moyenne : effort maîtrisé.
- Point d'inflexion : seuil critique.
- Complexité élevée : explosion de l'effort (maintenance, bugs).
- Conclusion : limiter la complexité, privilégier des designs simples et le refactoring.

3. L'histoire du « Bus Magique »

- Avec **n systèmes**, une communication point-à-point nécessite jusqu'à **$n \times (n - 1) / 2$ connexions** → explosion de la complexité.
- **Vision idéalisée :**
 - Un bus central unique (ESB) reliant tous les systèmes.
- **Réalité :**
 - Accumulation d'adaptateurs, de routes et de transformations.
 - Système difficile à maintenir.
- **Message clé :**
 - Les solutions centralisées cachent une forte complexité.
 - D'où l'intérêt d'architectures **décentralisées, légères** (microservices, messagerie asynchrone).

4. SOA (Service-Oriented Architecture)

- **Services :**
 - Fonctionnalités métier autonomes et réutilisables.
 - Indépendants des technologies et plateformes.
- **Infrastructure :**
 - ESB pour orchestrer et combiner les services.
- **Politiques et processus :**
 - Gestion de l'hétérogénéité, de la maintenance et des multiples acteurs.
- **Principe clé :**
 - Flexibilité, décentralisation et tolérance aux pannes.

5. SOA en cinq points

- **SOA :** paradigme pour gérer les processus métier dans de grands systèmes distribués.
- **Policies & Processes :**
 - La distribution impacte l'organisation et la collaboration entre équipes.
- **Web Services :**
 - Une implémentation possible de SOA, mais SOA ne se limite pas à eux.
- **SOA in Practice :**
 - Écart entre théorie et pratique (performance, sécurité, contraintes réelles).
- **Governance & Management :**
 - Élément central : bonnes règles, bonnes équipes, bonne gestion.

6. Terminologie SOA

- **Provider :**
 - Système qui expose un service.
- **Consumer :**
 - Système qui consomme le service.

7. Services en SOA

- **Fonctionnalité métier :**
 - Chaque service représente un processus métier élémentaire (EBP).
- **Interfaces et contrats :**
 - Définissent clairement comment utiliser le service.
- **Attributs clés :**
 - **Autonome** : fonctionne indépendamment.
 - **Coarse-grained** : granularité métier élevée.
 - **Visible / Discoverable** : facilement trouvable et utilisable.
 - **Stateless** : pas d'état entre les appels.
 - **Idempotent** : appels répétés sans effets indésirables.

Chapitre 7 : SOAP Web Services

1. Services Web et SOA

- Les **services web** permettent de créer des applications **interopérables** sur le Web.
- Ils sont fortement liés à **SOA**.
- Les échanges reposent principalement sur **XML**.

2. SOAP Web Service : fonctionnement

Un service Web SOAP suit un cycle standard :

1. **Définition** : le fournisseur décrit le service via un **WSDL**.
2. **Publication** : le WSDL est publié dans un annuaire **UDDI**.
3. **Recherche** : le client recherche le service dans l'annuaire.
4. **(Optionnel) Enregistrement** : le client s'enregistre auprès du fournisseur.
5. **Appel** : le client invoque le service via des **messages SOAP**.
6. **Composition** : les résultats peuvent être utilisés pour appeler d'autres services.

Résumé : *définir → publier → rechercher → appeler → composer*

3. Modèle Publish – Find – Bind (SOA)

- **Service Provider** : expose le service et publie le WSDL.
- **Service Requester** : client consommateur du service.
- **UDDI** : annuaire des services.
- Le client récupère le WSDL puis appelle directement le provider via SOAP.
- **Avantages** : couplage faible, découverte dynamique, interopérabilité.

4. Comparaison avec CORBA

- **CORBA** : middleware de communication entre objets distribués via un **ORB**.
- Appels distants transparents (comme des appels locaux).
- Interfaces décrites en **IDL**.
- Utilise des **stubs** (client) et **skeletons** (serveur).
- **Avantages** : multi-langages, forte abstraction.
- **Limites** : infrastructure lourde et complexe.
- **Idée clé** : CORBA est un ancêtre des solutions modernes (SOAP, REST, gRPC).

5. Technologies de base des Web Services

- **SOAP** : protocole d'échange de messages XML.
- **WSDL** : description du service (opérations, données, accès).
- **UDDI** : annuaire pour publier et découvrir les services.

6. SOAP

Définition

- **SOAP** est une norme W3C pour l'échange de messages entre applications.
- Versions : 1.0 → 1.1 → **1.2 (W3C)**.

Objectifs

- RPC sur le Web.
- Interopérabilité entre systèmes hétérogènes.
- Utilisation de XML et HTTP.
- Alternative plus adaptée au Web que CORBA.

Structure d'un message SOAP

- **Envelope** : conteneur XML.
- **Header (optionnel)** : sécurité, routage, transactions...
- **Body (obligatoire)** : données applicatives.
- **Fault** : gestion des erreurs.

7. En-tête et cheminement SOAP

- **Header** :
 - Métadonnées techniques.
 - Attributs : **mustUnderstand**, **relay**, **role**.
- Les nœuds intermédiaires :
 - Traitent les en-têtes qui les concernent.
 - Respectent **mustUnderstand**.
- En cas d'erreur → retour d'un **SOAP Fault**.

8. Appel RPC avec SOAP

- Deux messages :
 - **Requête** : nom de la méthode + paramètres.
 - **Réponse** : résultat ou Fault.
- Transport généralement via **HTTP**.

9. SOAP et HTTP

- **Binding SOAP** : définit comment SOAP utilise le protocole de transport.
- SOAP sur HTTP :
 - **GET** : seule la réponse est SOAP.
 - **POST (préféré)** : requête et réponse SOAP.
- Gestion des erreurs via les **codes HTTP**.

10. WSDL

Définition

- Langage XML décrivant un service web.
- Extension `.wsdl`.

Éléments principaux

- `<types>` : types de données.
- `<message>` : messages échangés.
- `<portType>` : opérations du service.
- `<binding>` : protocole et format (SOAP/HTTP).

Types d'opérations

- One-way
- Request-response
- Solicit-response
- Notification

11. UDDI

Définition

- **Annuaire universel** de services web.

Contenu d'une entrée

- **businessEntity** : fournisseur.
- **businessService** : services proposés.
- **bindingTemplate** : détails techniques.
- **tModel** : informations génériques.

Types d'informations

- **Pages blanches** : infos du fournisseur.
- **Pages jaunes** : classification du service.
- **Pages vertes** : informations techniques.